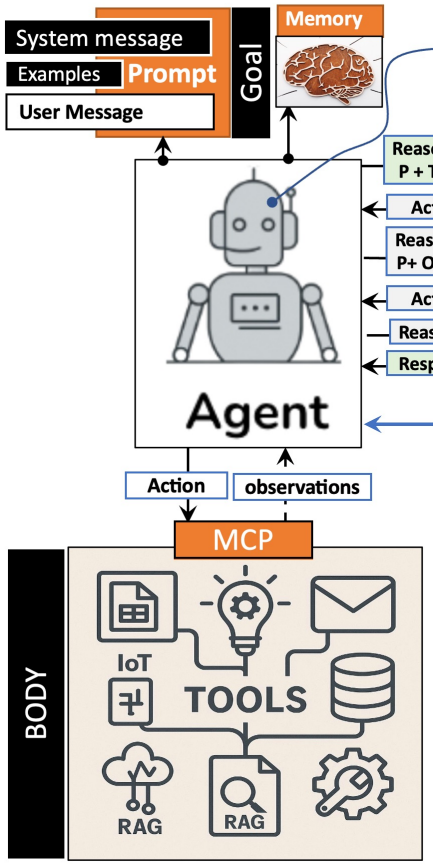
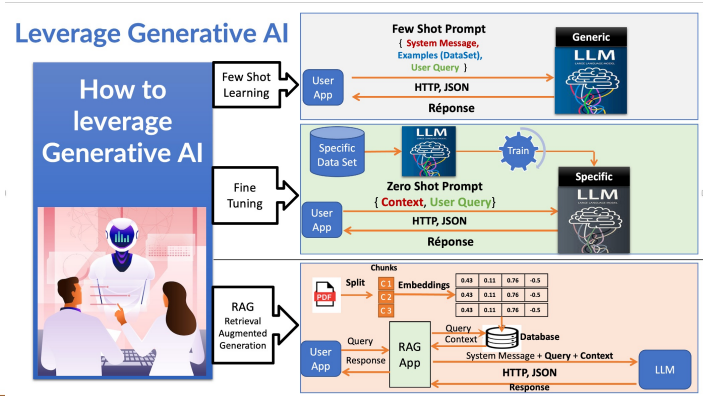
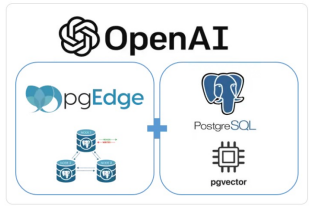
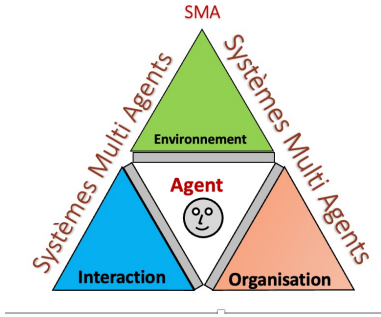
Agentic AI – Multi Agent Systems



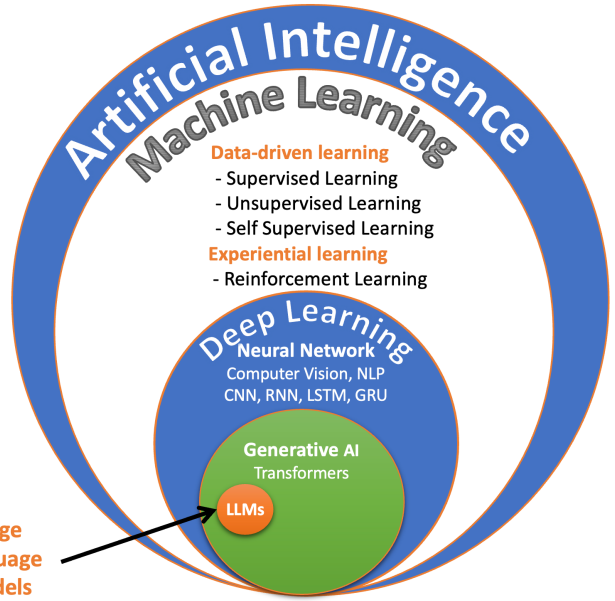










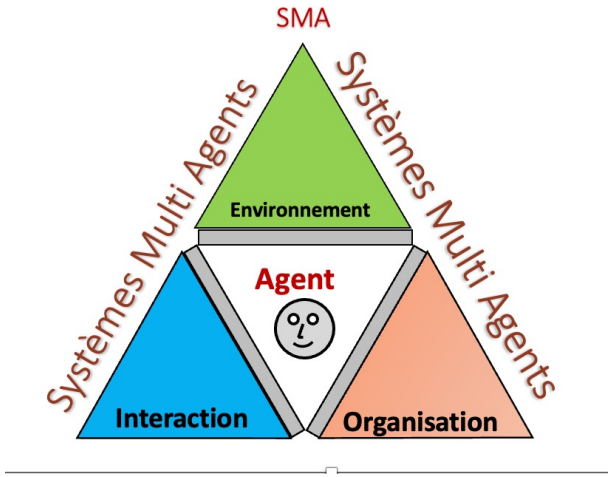


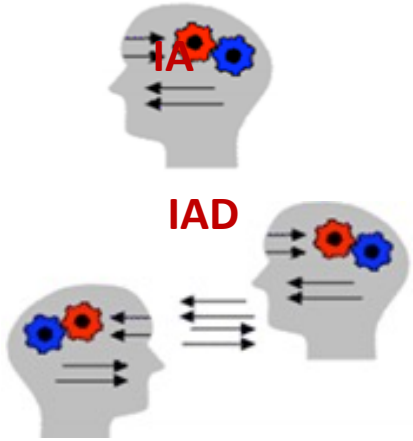
Intelligence Artificielle Distribuée & AI Agents

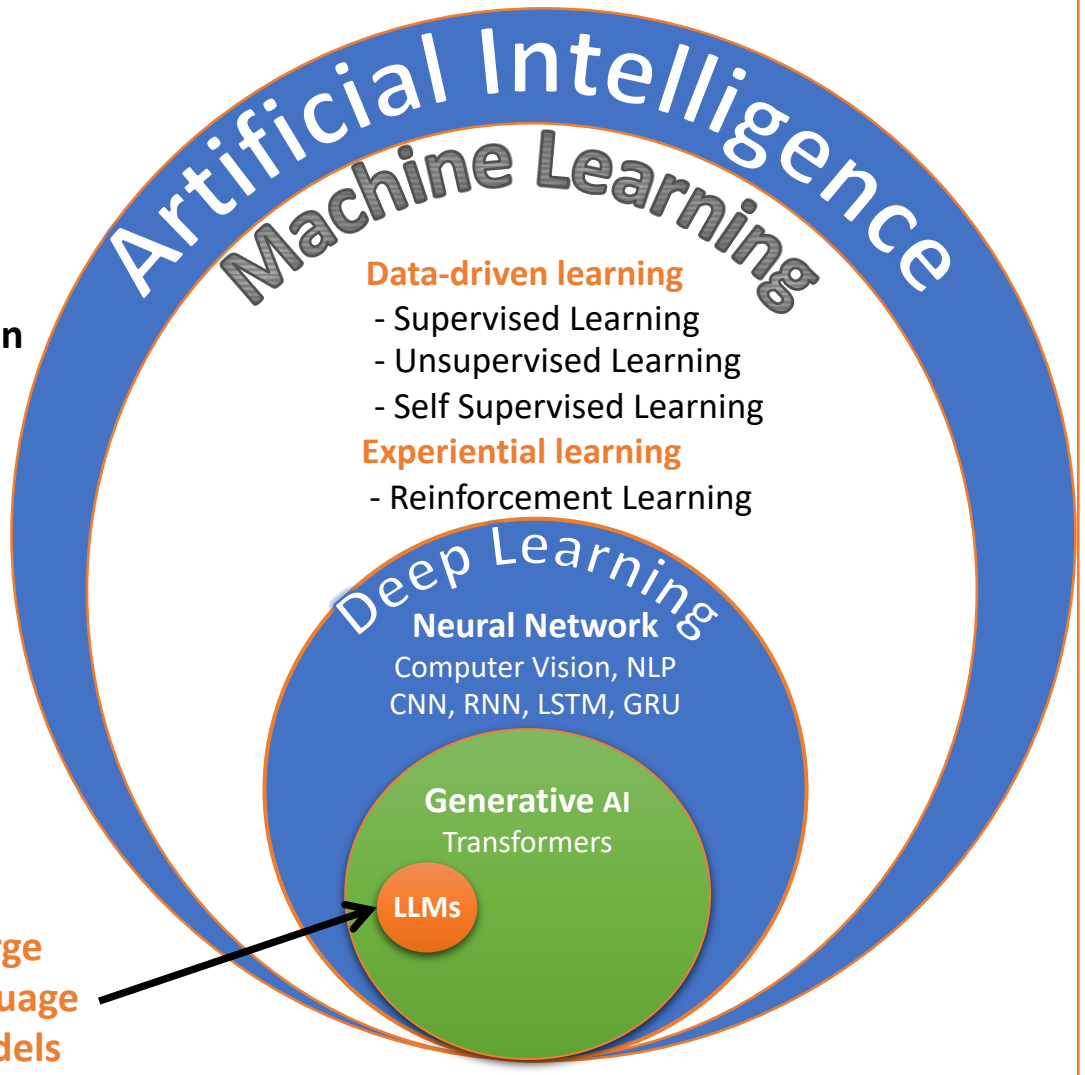
- L'**intelligence artificielle**
- **Intelligence Artificielle Distribuée** =
  - **IA** : Pour des agents intelligent (Modéliser le savoir et le comportement)
  - + **Distribuée** : Modéliser leurs interactions => **Intelligence Collective**
- **IA = IA symbolique (5%) + Machine Learning (95%)**
- **Techniques d'apprentissage :**
  - **Piloté par les données**
  - **Piloté par l'expérience**

→ **Apprentissage Supervisé** → **Régression** / **Classification**
→ **Apprentissage Non Supervisé**
→ **Apprentissage Auto Supervisé**

- **1985 : Systèmes Multi Agents (AI Agents)**







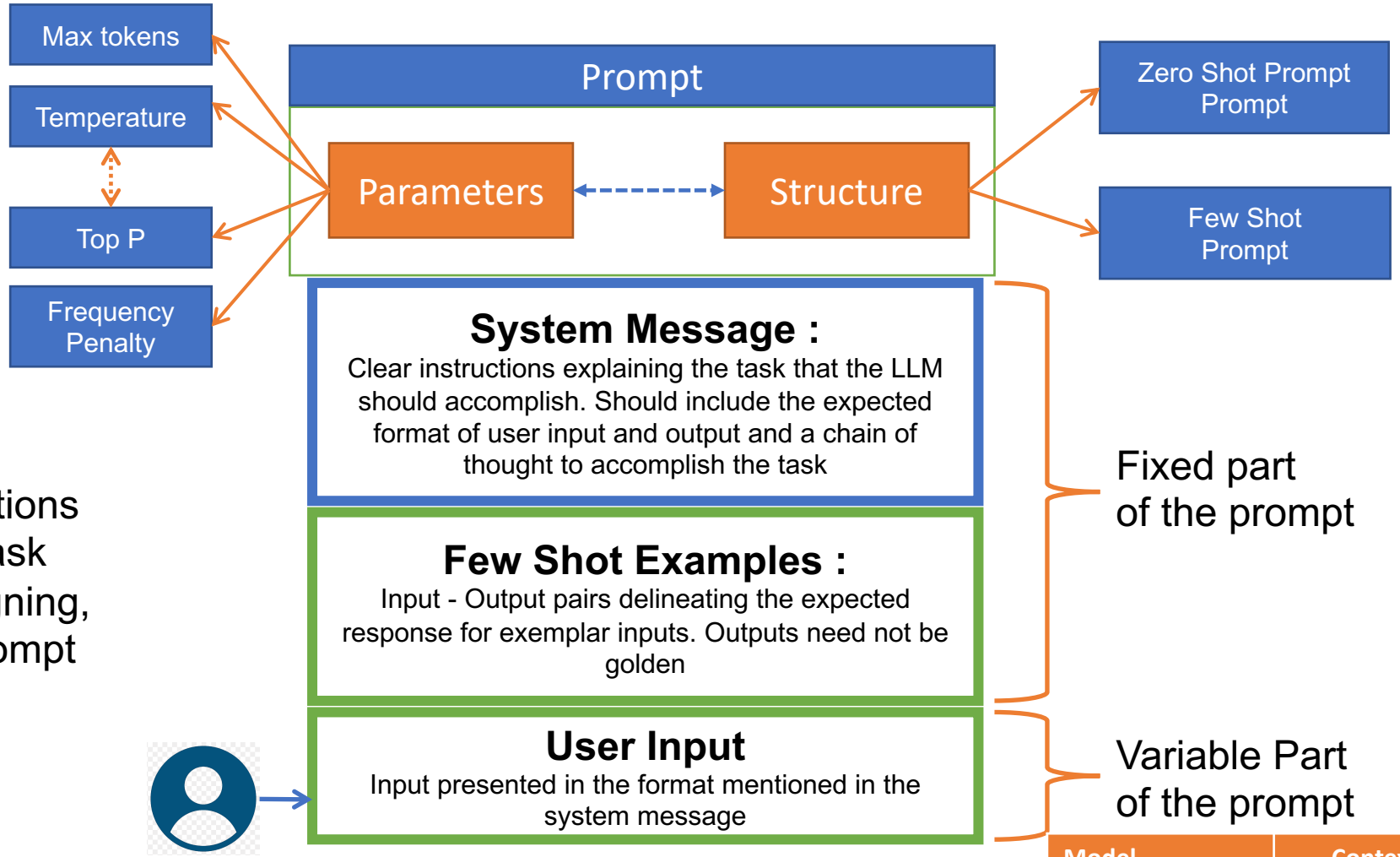
Prompt Engineering

Length of input + output

More temperature = More randomness in response

More Top P = More tokens selected for completion ; nucleus sampling échantillonnage du noyau

More FP = Less chance of tokens repeating



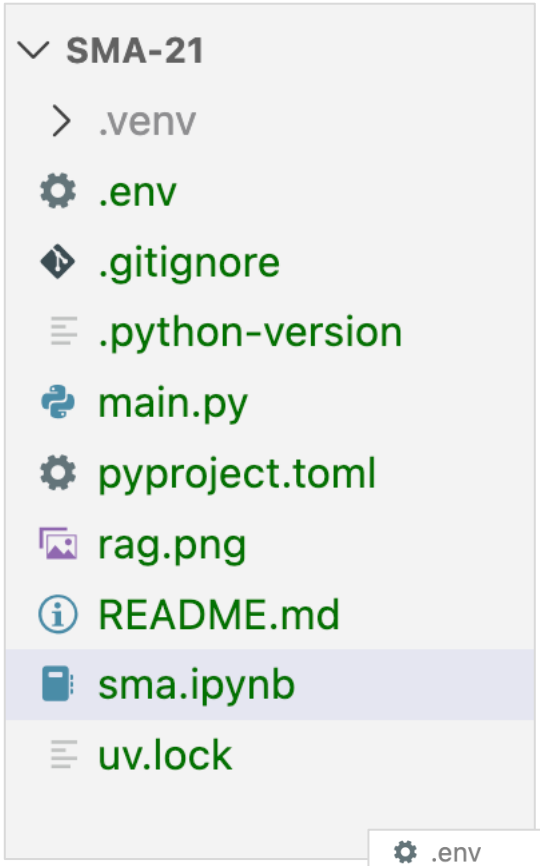
- **Prompt** = Specific set of instructions sent to a LLM to accomplish a task
- **Engineering** = Process of designing, evaluating and deploying the prompt for specific tasks
  1. Define Tasks and Metrics
  2. Data Set (Examples, Gold Examples)
  3. Design Prompts
  4. Evaluate Prompts
  5. Deploy and Monitor Prompt

https://platform.openai.com/tokenizer

https://platform.openai.com/

Model	Context Window
Gpt-3.5-turbo	4096
Gpt-4-tubo	128 000
Mixtral-8x7B	128 000

Prompt Engineering




```
$ python --version
  Python 3.11.3
$ uv --version
  uv 0.6.16 (d8ad9d3cd 2025-04-22)
$ uv init
$ uv venv
$ source ./.venv/bin/activate
sma-21) (base) $ uv add ipykernel
sma-21) (base) $ uv add tiktoken
sma-21) (base) $ uv add python-dotenv
sma-21) (base) $ uv add langchain
sma-21) (base) $ uv add langchain-openai
sma-21) (base) $ uv add langchain-ollama
sma-21) (base) $ uv add langchain-groq
```

pyproject.toml

```
[project]
name = "sma-21"
version = "0.1.0"
description = "Add your
description here"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
  "ipykernel>=7.2.0",
  "langchain>=1.2.10",
  "langchain-groq>=1.1.2",
  "langchain-ollama>=1.0.1",
  "langchain-openai>=1.1.9",
  "openai>=2.21.0",
   "python-dotenv>=1.2.1",
]
```

.env

```
1  OPENAI_API_KEY=sk-proj-gBJaJ8k_1sagyLqZpgi2C88zPvOM
2  GROQ_API_KEY=gsk_a2RhgQGzK8zxwV4oqnKAWGdyb3FYkIQOF1
```

```
> .\.venv\Scripts\activate
   > Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
```

Prompt Engineering (Tokenizer : Tiktoken)

```
import tiktoken
```

```
#encoding = tiktoken.get_encoding("cl200k_base")
encoding = tiktoken.encoding_for_model("gpt-4o")
print(encoding.name)
system_message ="""
Perform Sentiment analysis of the review presented in
the user message.
The result should be positive or negative. Do not
justify your response
"""
tokens = encoding.encode(system_message)
```

Encoding name	OpenAI models
`o200k_base`	gpt-4o, gpt-4o-mini, gpt-5
`cl100k_base`	gpt-4-turbo, gpt-4, gpt-3.5-turbo, text-embedding-ada-002, text-embedding-3-small, text-embedding-3-large
`p50k_base`	Codex models, `text-davinci-002`, `text-davinci-003`
`r50k_base` (or `gpt2`)	GPT-3 models (e.g., `davinci`)

o200k_base

```
def num_tokens_from_string(string: str, encoding_name: str = "o200k_base") -> int:
"""Returns the number of tokens in a text string."""
encoding = tiktoken.get_encoding(encoding_name)
num_tokens = len(encoding.encode(string))
return num_tokens
```

```
print(len(tokens))
print(tokens)
for token in tokens:
print(encoding.decode_single_token_bytes(token=token),end="")
```

```
num_tokens_from_string("tiktoken is great!")
```

6

28

[793, 73751, 31712, 2581, 8450, 328, 290, 3358, 13392, 306, 290, 1825, 3176, 558, 976, 1534, 1757, 413, 8841, 503, 12626, 13, 3756, 625, 14883, 634, 3322, 198]

b' \n'b'Perform'b' Sent'b'iment'b' analysis'b' of'b' the'b' review'b' presented'b' in'b' the'b' user'b' message'b'.\n'b'The'b' result'b' should'b' be'b' positive'b' or'b' negative'b'.'b' Do'b' not'b' justify'b' your'b' response'b'\n'

Prompt Engineering (Interagir avec un LLM OpenAI)

```
from langchain_openai import ChatOpenAI
from dotenv.ipython import load_dotenv
from IPython.display import Markdown
import os
```

```
load_dotenv(override=True)
```

```
# os.environ['OPENAI_API_KEY’]=« ...."
```

```
model = ChatOpenAI(
  model="gpt-5.2"
)
```

Un **Agent AI** (ou **agent d’intelligence artificielle**) est un programme capable de **percevoir une situation**, **prendre des décisions** et **agir** pour atteindre un objectif, souvent de façon **semi-autonome** ou **autonome**.

Ce qui le distingue d’un simple chatbot

- Un **chatbot** répond surtout à des questions.
- Un **agent AI** peut aussi **planifier**, **utiliser des outils** (API, web, fichiers, calendrier, base de données), **exécuter des actions**, puis **vérifier le résultat** et s’adapter.

Comment ça fonctionne (schéma simple)

1. **Objectif** : “Réserver un billet”, “Automatiser un rapport”, “Répondre au support client”.
2. **Observation / contexte** : données utilisateur, état d’un système, documents, emails, etc.
3. **Raisonnement / planification** : découpe le but en étapes.
4. **Actions** : appelle des outils (recherche, calcul, envoi d’email, création de ticket…).
5. **Boucle de feedback** : vérifie si ça marche, corrige, recommence si besoin.

Exemples concrets

- **Agent de support client** : lit un ticket, cherche dans la base de connaissances, propose une réponse, crée/assigne un ticket si nécessaire.
- **Agent “assistant de réunion”** : résume un échange, extrait des actions, planifie des tâches, envoie un compte rendu.
- **Agent de dev (coding agent)** : ouvre un repo, propose un correctif, lance des tests, fait une PR.

Points importants

- **Autonomie variable** : parfois il demande confirmation avant d’agir (mode “copilote”), parfois il agit seul.
- **Risques** : erreurs, actions non désirées, sécurité des données → on met souvent des **garde-fous** (permissions, validation, logs).

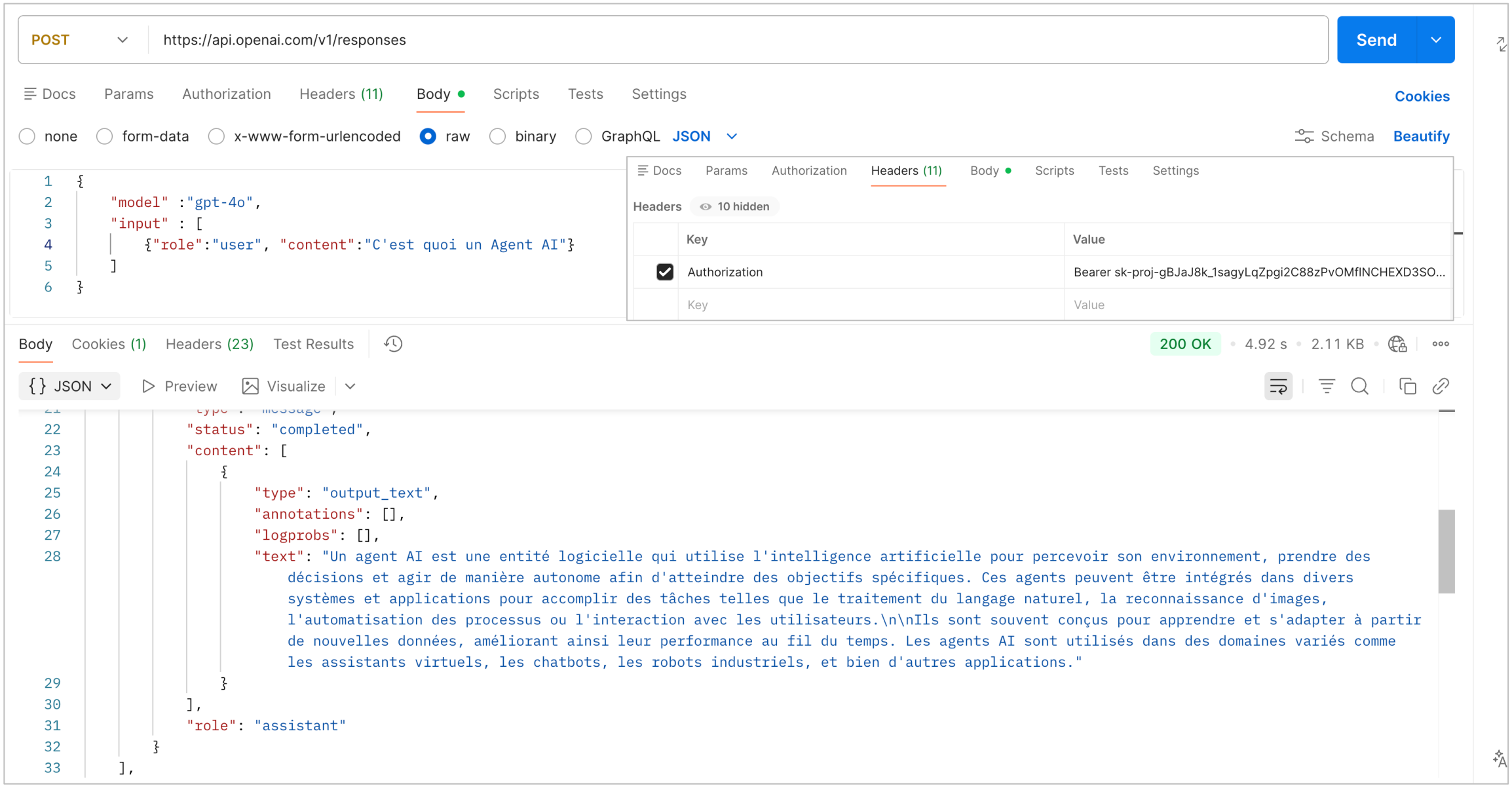
Si tu me dis ton contexte (pro, perso, une app que tu veux construire), je peux te proposer un exemple d’agent AI adapté.

None

```
response = model.invoke([
 {"role":"system", "content":"You are a helpful assistant. The output should be in Markdown"},
 {"role":"user","content":"C'est quoi un Agent AI"}
])
```

```
print(display(Markdown(response.content)))
```

Prompt Engineering (Interagir avec un LLM OpenAI avec Postman)



Prompt Engineering (Interagir avec un LLM local Ollama)

```python
from langchain_ollama import ChatOllama
```

```python
llm = ChatOllama(model="qwen3-vl:235b-cloud")
```

```python
response = llm.invoke([
 {"role":"system", "content":"You are a helpful assistant. The output should be in Markdown"},
 {"role":"user","content":"C'est quoi l'ENSET"}
])
```

```python
print(display(Markdown(response.content)))
```

Qu'est-ce qu'un Agent AI ?

Un **Agent AI** (ou **Agent IA** en français) est un programme informatique ou un système capable d'agir de manière **autonome** dans un environnement pour atteindre des objectifs prédéfinis. Il perçoit son environnement via des capteurs (ou des données d'entrée), prend des décisions et exécute des actions pour influencer cet environnement.

Caractéristiques principales :

- **Autonomie** : Fonctionne sans intervention humaine constante.
- **Perception** : Analyse des données (texte, images, capteurs, etc.).
- **Prise de décision** : Utilise des règles, des algorithmes ou de l'apprentissage automatique.
- **Objectif orienté** : Agit pour maximiser une récompense ou atteindre un but.

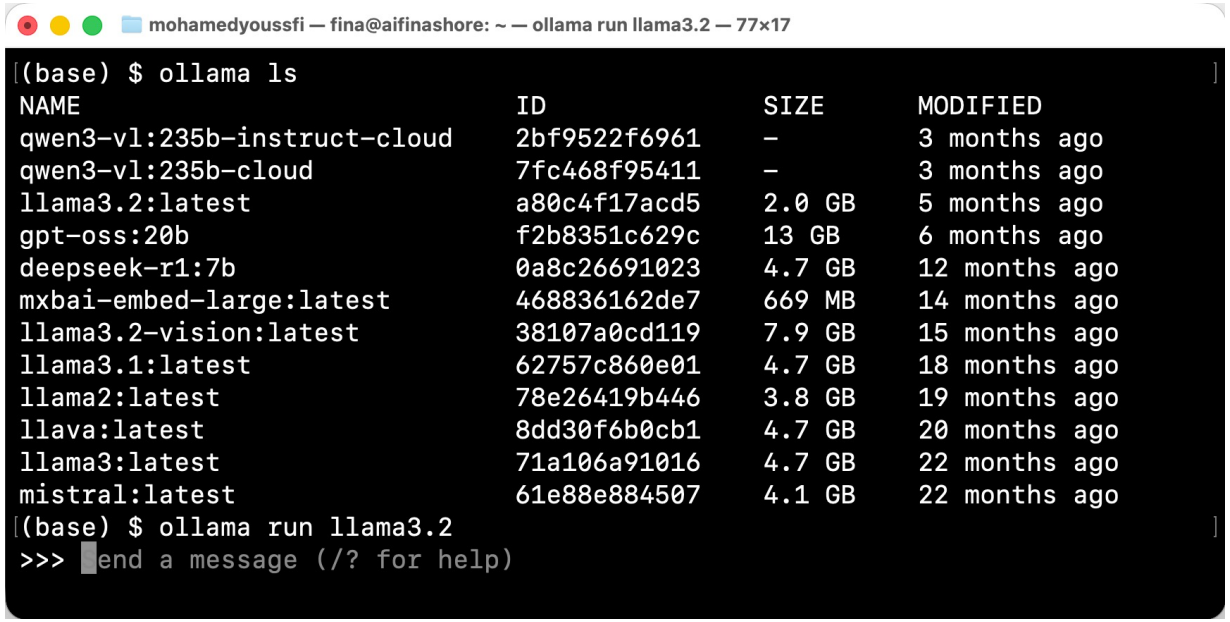
Exemples concrets :

- **Assistants virtuels** : Siri, Alexa, Google Assistant.
- **Chatbots** : Outils de service client en ligne (comme celui-ci !).
- **Systèmes de recommandation** : Netflix, Spotify, Amazon.
- **Robots autonomes** : Aspirateurs intelligents (Roomba), véhicules sans conducteur.

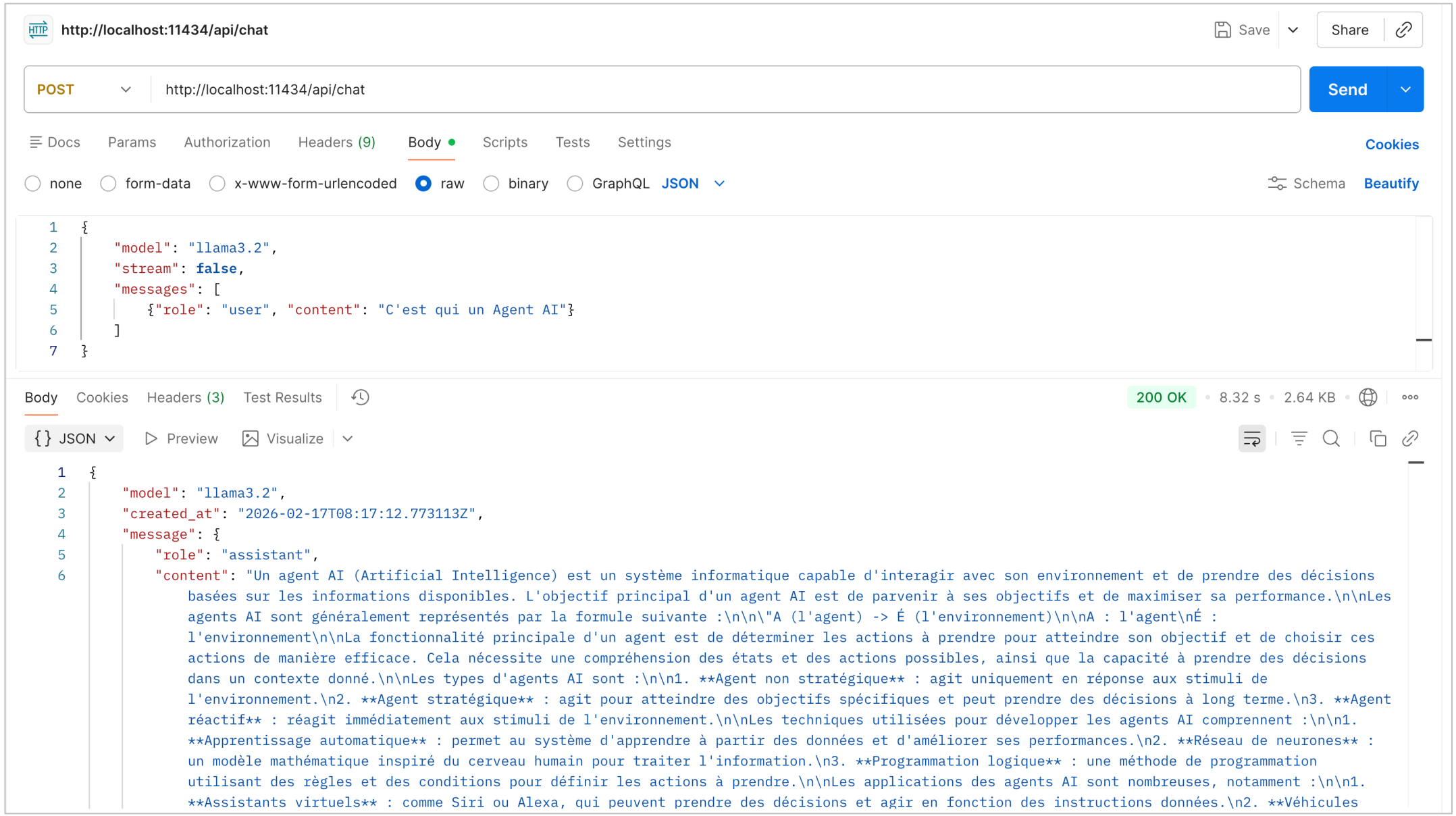
Types d'agents IA :

- **Agents réactifs** : Réagissent en temps réel à leur environnement (ex : thermostats intelligents).
- **Agents à but** : Planifient des actions pour atteindre un objectif (ex : systèmes de navigation GPS).
- **Agents d'apprentissage** : S'adaptent grâce à des données (ex : algorithmes de personnalisation).

En résumé, un Agent AI est un outil puissant qui combine **perception**, **raisonnement** et **action** pour automatiser des tâches complexes, améliorer l'efficacité ou offrir des expériences personnalisées.



Prompt Engineering (Interagir avec un LLM local Ollama avec Postman)



Prompt Engineering

```
from langchain_groq import ChatGroq
```

```
llm3 = ChatGroq(model="openai/gpt-oss-120b")
```

```
response = llm3.invoke([
 {"role":"system", "content":"You are a helpful assistant. The output should be in Markdown"},
 {"role":"user","content":"C'est quoi un Agent AI"}
])
```

```
print(display(Markdown(response.content)))
```

Qu'est-ce qu'un Agent IA ?

Un **Agent IA** (ou *agent d'intelligence artificielle*) est un système logiciel ou robotique capable d'agir **de manière autonome** pour atteindre des objectifs spécifiques, en utilisant des techniques d'intelligence artificielle. Contrairement à un simple programme, un agent IA perçoit son environnement, prend des décisions et agit sans intervention humaine continue.

Caractéristiques clés

- **Autonomie** : Fonctionne sans supervision constante.
- **Perception** : Collecte des données via des capteurs, APIs ou interfaces (ex : caméra, micro, données en temps réel).
- **Prise de décision** : Utilise des algorithmes (apprentissage automatique, logique symbolique, etc.) pour analyser les données.
- **Action** : Exécute des tâches (répondre à une question, commander un robot, ajuster un système).

Exemples concrets

- **Assistants virtuels** : Siri, Alexa, Google Assistant.
- **Chatbots** : Services client automatisés sur les sites web.
- **Robots autonomes** : Véhicules sans conducteur, drones de livraison.
- **Agents de trading** : Algorithmes analysant les marchés financiers en temps réel.

Comment ça fonctionne ?

1. **Perception** : L'agent récupère des données (ex : une commande vocale).
2. **Traitement** : Utilise des modèles d'IA (comme le NLP pour comprendre le langage naturel).
3. **Décision** : Choisit l'action optimale (ex : répondre à une question ou déclencher une action).
4. **Action** : Exécute la tâche (ex : lire une météo, contrôler un thermostat).

Domaines d'application

- **Santé** : Diagnostic médical assisté par IA.
- **E-commerce** : Recommandations personnalisées.
- **Industrie** : Maintenance prédictive des machines.
- **Jeux vidéo** : Personnages non-joueurs (PNJ) intelligents.

Défis

- **Éthique** : Biais dans les décisions, protection des données.
- **Limites techniques** : Complexité des environnements réels.
- **Confiance** : Acceptation par les utilisateurs face à l'autonomie de l'agent.

Un Agent IA est donc un **acteur proactif** de l'IA, conçu pour simplifier, automatiser ou optimiser des processus dans divers secteurs.

Image Generation

```
llm4 = ChatOpenAI(model="gpt-5.2")
llm_with_tools = llm4.bind_tools([
   {"type":"image_generation", "quality":"high"}
])
```

```
from langchain.messages import HumanMessage
```

```
resp4 = llm_with_tools.invoke(input=[
   HumanMessage("Je veux une photo d'un chat qui code du java")
])
```

```
from IPython.display import Image
import base64
```

```
Image(base64.b64decode(resp4.content_blocks[0]['base64']))
```

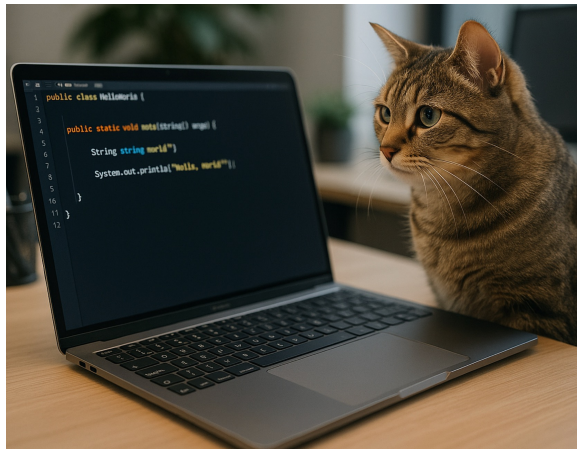
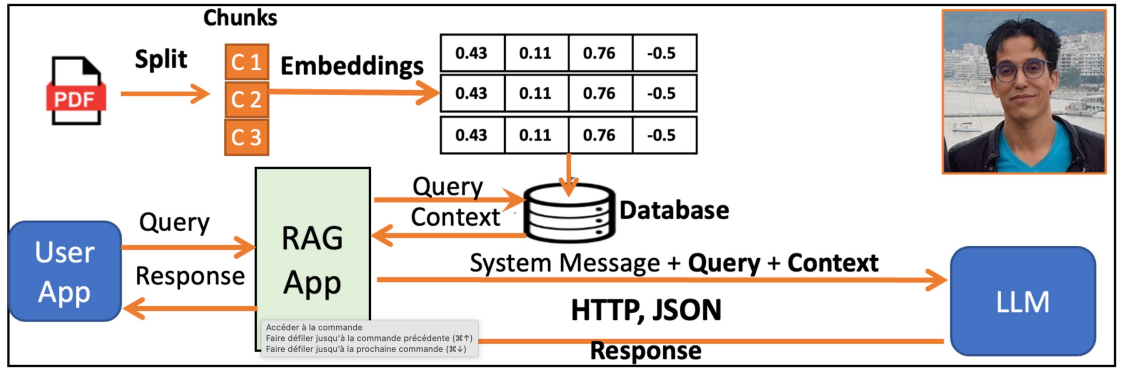




Image Description

```
path ="rag.png"
```

```
Image(path)
```




```
print(display(Markdown(resp5.content)))
```

Je vois un schéma qui explique le fonctionnement d'une application **RAG (Retrieval-Augmented Generation)** :

- Un **PDF** est **découpé (split)** en morceaux (*chunks* C1, C2, C3).
- Chaque chunk est transformé en **embeddings** (vecteurs de nombres).
- Ces embeddings sont stockés dans une **base de données** (type base/vector store).
- L'**utilisateur** envoie une **requête** via une *User App* vers la **RAG App**.
- La RAG App interroge la base pour récupérer le **contexte** pertinent (*Query Context*).
- Puis elle envoie au **LLM** (via **HTTP/JSON**) : **System Message + Query + Context**.
- Le LLM renvoie une **réponse**, qui repart vers l'app utilisateur.

En haut à droite, il y a aussi la **photo d'une personne** (portrait). Je ne peux pas l'identifier, mais je peux décrire l'image si tu veux.

```
llm5 = ChatOpenAI(model="gpt-5.2")
```

```
# Function to encode the image
def encode_image(image_path):
   with open(image_path, "rb") as image_file:
       return base64.b64encode(image_file.read()).decode("utf-8")
```

```
img = encode_image(path)
```

```
resp5 = llm5.invoke(input=[
HumanMessage(content=[
{"type":"text","text":"Qu'est ce que tu vois dans cette image"},
{"type":"image_url", "image_url": {"url": f"{img}"}}
])])
```

Prompt Engineering (Exemple Analyse du sentiment)

```
system_message = """
Effectuez une analyse de sentiments basée sur les aspects des avis concernant les ordinateurs portables présentés en entrée.
Chaque avis peut comporter un ou plusieurs des aspects suivants : screen, keybloard et pad.
Pour chaque avis présenté en entrée :
- Identifiez la présence d'au moins un des trois aspects (screen, keybloard, pad).
- Attribuez une polarité de sentiment (positive, negative ou neutral) à chaque aspect. Organisez votre réponse dans un objet
JSON avec les en-têtes suivants :
- category:[liste des aspects]
- polarity:[liste des polarités correspondantes pour chaque aspect]
Si l'un des aspects n'est pas présent dans l'avis de l'utilisateur, tu supposes que la polarité est neutre
"""
```

```
zeroshot_prompt = [
  {"role": "system", "content": system_message},
  {"role": "user", "content": """L'écran est très bon, mais je n'ai pas aimé la souris. le clavier Ma fih Maytchaf
    """}
 ]
```

```
llm = ChatOpenAI(model="gpt-5.2")
resp = llm.invoke(input=zeroshot_prompt)
```

```
print(resp.content)
```

```json
{
  "category": ["screen", "keybloard", "pad"],
  "polarity": ["positive", "negative", "negative"]
}
```

```
resp_text_json = resp.content.replace("```json","").replace("```","")
```

```
import json
```

```
result = json.loads(resp_text_json)
```

```
print(result['category'])
```

```
['screen', 'keybloard', 'pad']
```

```
print(result['polarity'][0])
```

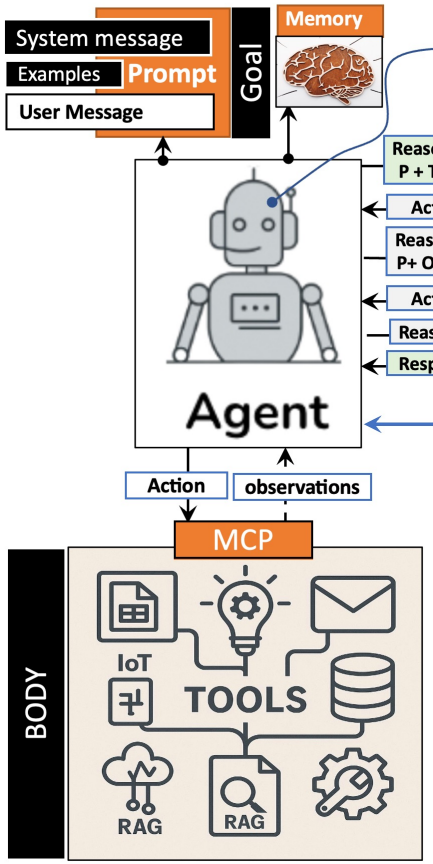
```
print(result['polarity'][0])
```

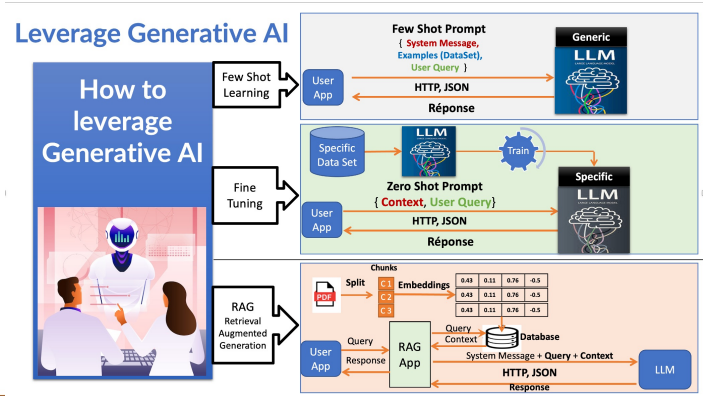
```
positive
```

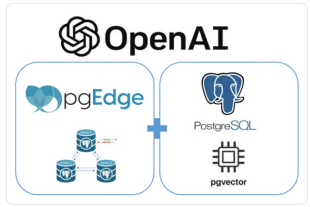
Activité Pratique N°1
Prompt Engineering

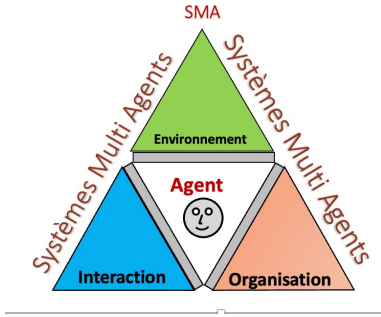
Agentic AI – Multi Agent Systems



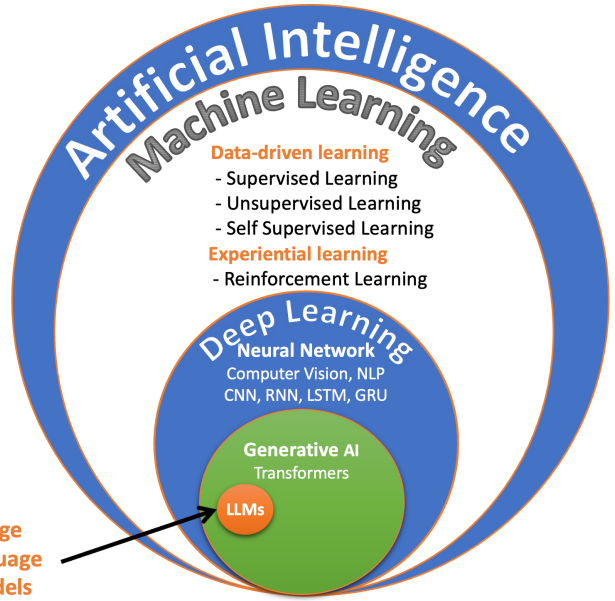












Use Case : Sentiment Analysis

- **Step 1 : Define Objectives and Metrics**
  - Dans l'analyse de sentiment, nous attribuons au texte fourni en entrée l'une des deux étiquettes suivantes : **positif** ou **négatif** (dans certains cas, l'une des trois étiquettes : **positif**, **négatif** ou **neutre**).
  - À titre d'exemple, nous allons attribuer un sentiment aux avis clients sur des produits répertoriés dans la catégorie habillement sur Amazon.
  - Étant donné que notre prédiction correspondra à l'une des deux étiquettes — **positif** ou **négatif** — nous pouvons utiliser les métriques suivantes pour mesurer la performance d'un prompt :
    - Taux d'exactitude (**accuracy**): calculé comme le nombre de prédictions correctes (c'est-à-dire le nombre de fois où la prédiction correspond à la vérité terrain / étiquette de référence).
    - **Score micro-F1** : calculé en agrégeant les **vrais positifs (TP)**, **faux négatifs (FN)** et **faux positifs (FP)** sur l'ensemble des classes. Cette métrique est utilisée lorsque nous ne soupçonnons pas de déséquilibre de classes (par exemple, si nous nous attendons à ce qu'il n'y ait pas une quantité écrasante d'avis positifs ou négatifs).
  - En pratique, il est préférable d'utiliser une métrique nuancée telle que le **micro-F1** plutôt que la simple **accuracy**.

Pour illustrer le calcul du **micro-F1** dans un problème de **classification binaire**, on s'appuie sur une **matrice de confusion**, qui comptabilise les **vrais positifs (TP)**, **faux positifs (FP)**, **vrais négatifs (TN)** et **faux négatifs (FN)**.

Exemple de matrice de confusion :

1) Totaux (micro)

- **TP total** = 50
- **FP total** = 10
- **FN total** = 5

Prédit \ Réel	Positif	Négatif
Positif	50	10
Négatif	5	100

2) Calculs (micro)

- **Micro-précision** = TP / (TP + FP) = 50 / (50 + 10) = **0,833**
- **Micro-rappel (recall)** = TP / (TP + FN) = 50 / (50 + 5) = **0,909**
- **Micro-F1** = 2 × (Précision × Rappel) / (Précision + Rappel) = 2 × (0,833 × 0,909) / (0,833 + 0,909) = **0,870**

Conclusion : le score micro-F1 de cet exemple est 0,870.

Dans ce notebook, nous utiliserons l'implémentation `f1_score` de `scikit-learn` pour évaluer les prompts d'analyse de sentiment.

Step 2 : Assemble Data

```
import session_info
import pandas as pd
from tqdm import tqdm
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
import numpy as np
from datasets import load_dataset
from dotenv.ipython import load_dotenv
import session_info
import matplotlib.pyplot as plt
```

```
load_dotenv(override=True)
```

✓ 0.0s

True

```
session_info.show()
```

✓ 0.2s

▼ Click to view session information

```
-----
datasets                    4.5.0
dotenv                      NA
langchain_community         0.4.1
langchain_core              1.2.13
langchain_openai            NA
langchain_text_splitters    NA
matplotlib                  3.10.8
numpy                       2.4.2
pandas                      3.0.1
session_info                v1.0.1
sklearn                     1.8.0
tiktoken                    0.12.0
tqdm                        4.67.3
-----
```

▶ Click to view modules imported as dependencies

```
-----
IPython             9.10.0
jupyter_client      8.8.0
jupyter_core        5.9.1
-----
Python 3.11.3 (v3.11.3:f3909b8bc8, Apr  4 2023, 20:
macOS-26.2-arm64-arm-64bit
-----
Session information updated at 2026-02-19 21:52
```

Step 2 : Assemble Data

- Pour l'exercice d'analyse de sentiment, nous utiliserons un jeu de données d'avis de films [collectés (scrapés) depuis IMDB] (https://huggingface.co/datasets/imdb).
- Notre étude se concentrera sur l'attribution d'un sentiment **positif** ou **négatif** aux avis de films que les utilisateurs ont publiés sur IMDB. Lors de l'ingénierie de prompts, nous utiliserons un **jeu de données de validation** (hold-out) d'avis (c'est-à-dire des **exemples de référence / gold examples**) afin d'évaluer la qualité de l'attribution du sentiment.

```
corpus = load_dataset("imdb")
```

✓ 4.6s

```
corpus
```

✓ 0.0s

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
```

Step 2 : Assemble Data

- Notez que le jeu de données est déjà divisé en trois sous-ensembles : **train**, **test** et **unsupervised** (c'est-à-dire sans étiquette). Les jeux de données d'apprentissage automatique sont généralement stockés de cette manière ; toutefois, nous n'avons pas l'intention d'entraîner un classifieur de sentiment à partir de zéro, nous nous concentrons donc uniquement sur les données d'entraînement.
- Lorsque nous constituons un jeu de données destiné à être utilisé avec un **LLM**, nous créons des partitions pour les **exemples** et les **exemples de référence (gold examples)**, puis nous enregistrons ces versions pour de futures itérations.
- Voyons comment ce processus est appliqué au sous-ensemble `**train**` des données. Pour faciliter la préparation des données, convertissons `**train**` en **DataFrames**.
- Remarquez que le jeu de données est parfaitement équilibré, avec un nombre égal d'avis associés au sentiment **positif** (label 1) et au sentiment **négatif** (label 0).

```
train_df = corpus['train'].to_pandas()
```
✓ 0.0s

```
train_df.info()
```
✓ 0.0s

```
<class 'pandas.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   text    25000 non-null  str
 1   label   25000 non-null  int64
dtypes: int64(1), str(1)
memory usage: 32.0 MB
```

```
train_df.label.value_counts()
```

```
label
0    12500
1    12500
Name: count, dtype: int64
```

Step 2 : Assemble Data

- Afin de faciliter la comparaison des étiquettes de référence (ground truth) avec les prédictions du modèle, ajoutons l'étiquette textuelle correspondante comme colonne supplémentaire dans les données.
- Nous disposons désormais du jeu de données au format adéquat pour être traité par un LLM.

```
train_df['sentiment'] = np.where(train_df.label == 1, "positive", "negative")
```

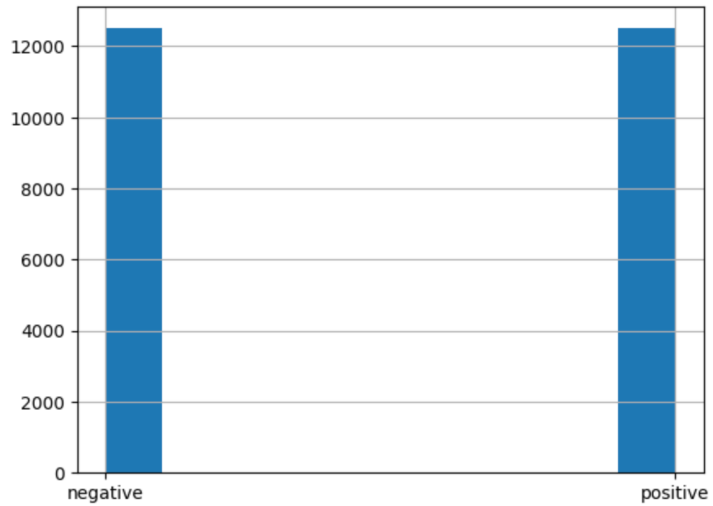
```
train_df.sample(6)
```

	text	label	sentiment
7557	True, there are many movies much worse then th...	0	negative
160	Not a `woman film' but film for the gang. One ...	0	negative
15995	Guys and Dolls has to be one of my favorite mu...	1	positive
1716	Watching That Lady In Ermine I was wondering w...	0	negative
7680	I picked this DVD up at the Dollar Store. The ...	0	negative
9959	I saw this movie way back at the first theatri...	0	negative

```
train_df.sentiment.value_counts()
```

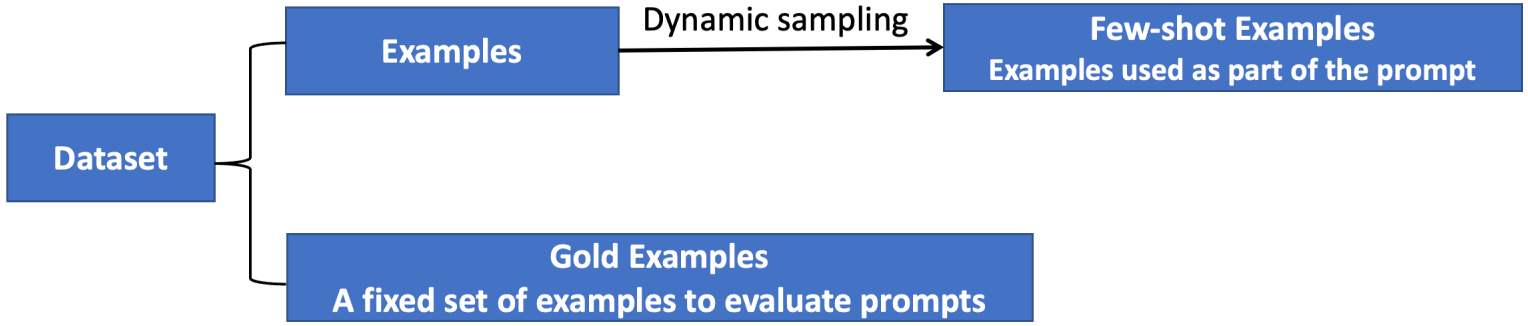
```
sentiment
negative    12500
positive    12500
Name: count, dtype: int64
```

```
train_df.sentiment.hist()
```



Step 2 : Assemble Data

- Maintenant que le prétraitement est terminé, divisons les données en deux segments : un segment constituant un réservoir dans lequel puiser des exemples ***few-shot***, et un autre segment constituant un réservoir d'exemples de référence (***gold examples***).
- En résumé (voir la figure),
  - Nous extrayons un jeu de données à partir d'un corpus en traitant les champs requis.
  - Chaque exemple doit contenir le texte d'entrée et une étiquette annotée.
  - Une fois que nous avons créé des exemples et des exemples de référence (**gold examples**) à partir de ce jeu de données, cet ensemble de données curé est enregistré dans un format adapté à la réutilisation (par exemple, **JSON**).




```
examples_df, gold_examples_df = train_test_split(
    train_df, test_size=0.2, random_state=42
)
```

```
(examples_df.shape, gold_examples_df.shape)
```

```
((20000, 3), (5000, 3))
```

Step 2 : Assemble Data

- Pour sélectionner des exemples de référence (**gold examples**) pour cette session,
  - Nous effectuons un échantillonnage aléatoire à partir des données de test en utilisant `random_state=42`.
  - Cela garantit que les exemples obtenus lors de plusieurs exécutions de l'échantillonnage sont identiques (c'est-à-dire qu'ils sont sélectionnés aléatoirement, mais ne changent pas d'une exécution à l'autre du notebook).
  - Notez que nous faisons cela uniquement pour maintenir des temps d'exécution faibles à des fins d'illustration.
  - En pratique, un grand nombre d'exemples de référence permet d'obtenir des estimations plus robustes de la précision du modèle.

```
columns= ['text', 'sentiment']
gold_examples = (
    gold_examples_df.loc[:, columns]
    .sample(20, random_state=42)
    .to_json(orient='records'))
```

✓ 0.0s

```
json.loads(gold_examples)[0]
```

```
{'text': 'Like I said at the top, four star
 'sentiment': 'positive'}
```

Étape 3 : Élaborer le prompt

Prompt 1: Zero-shot

```
zero_shot_system_message = """
Classify the sentiment of movie reviews presented in the input as 'positive' or 'negative'
Movie reviews will be delimited by triple backticks in the input.
Answer only 'positive' or 'negative'
Do not explain your answer.
"""
```

Python

```
zero_shot_prompt = [{'role':'system', 'content': zero_shot_system_message}]
```

Python

Étape 3 : Élaborer le prompt

- Pour le prompt ***few-shot***, le message système ne change pas par rapport au prompt ***zero-shot***.
- En revanche, nous enrichissons ce message système avec des exemples ***few-shot***.
- Pour constituer des exemples **few-shot**, nous devons échantillonner le nombre d'avis requis à partir des données d'entraînement.
- Une approche consiste à commencer par isoler les avis positifs et négatifs, puis à sélectionner des échantillons dans ces sous-ensembles, comme suit (remarquez que nous constituons 8 exemples **few-shot** dans le code ci-dessous) :

```
few_shot_system_message = """
Classify the sentiment of movie reviews presented in the input as
'positive' or 'negative'
Movie reviews will be delimited by triple backticks in the input.
Answer only 'positive' or 'negative'
Do not explain your answer.
"""
```

```
positive_reviews = (examples_df.sentiment == 'positive')
negative_reviews = (examples_df.sentiment == 'negative')
```

```
(positive_reviews.shape, negative_reviews.shape)
```

```
((20000,), (20000,))
```

```
columns = ['text', 'sentiment']
positive_examples = examples_df.loc[positive_reviews, columns].sample(4)
negative_examples = examples_df.loc[negative_reviews, columns].sample(4)
```

```
positive_examples
```

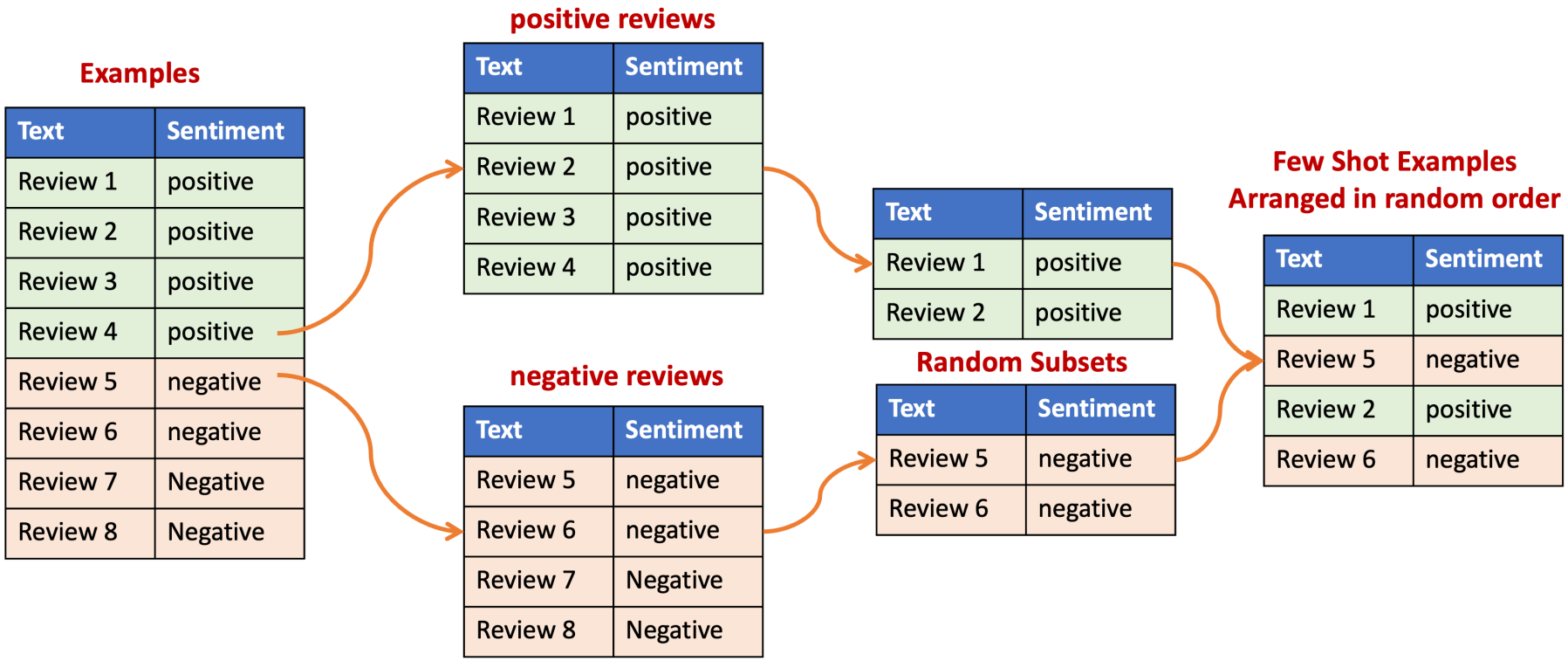
	text	sentiment
12746	After some of the negative reviews i heard on ...	positive
12756	This film captured my heart from the very begi...	positive
18841	This movie (and yes, it's a movie - it was sho...	positive
14640	This film was an interesting take by Hollywood...	positive

```
negative_examples
```

	text	sentiment
6844	Feh. This movie started out in an interesting ...	negative
158	SPOILER: The young lover, Jed, is kicked out b...	negative
2801	Singleton has some serious issues he has to co...	negative
11716	This 2003 made for TV movie was shown on a wom...	negative

Étape 3 : Élaborer le prompt

- Cependant, le simple fait de sélectionner des échantillons aléatoires à partir des sous-ensembles de polarité ne suffit pas, car les exemples inclus dans un prompt sont sujets à un ensemble de biais connus, tels que :
  - **Biais de l'étiquette** majoritaire** (réponses les plus fréquentes dans les prédictions)
  - **Biais de récence** (exemples situés vers la fin du prompt)
- Pour éviter ces biais, il est important de disposer d'un ensemble d'exemples équilibré et organisé dans un ordre aléatoire.



Étape 3 : Élaborer le prompt

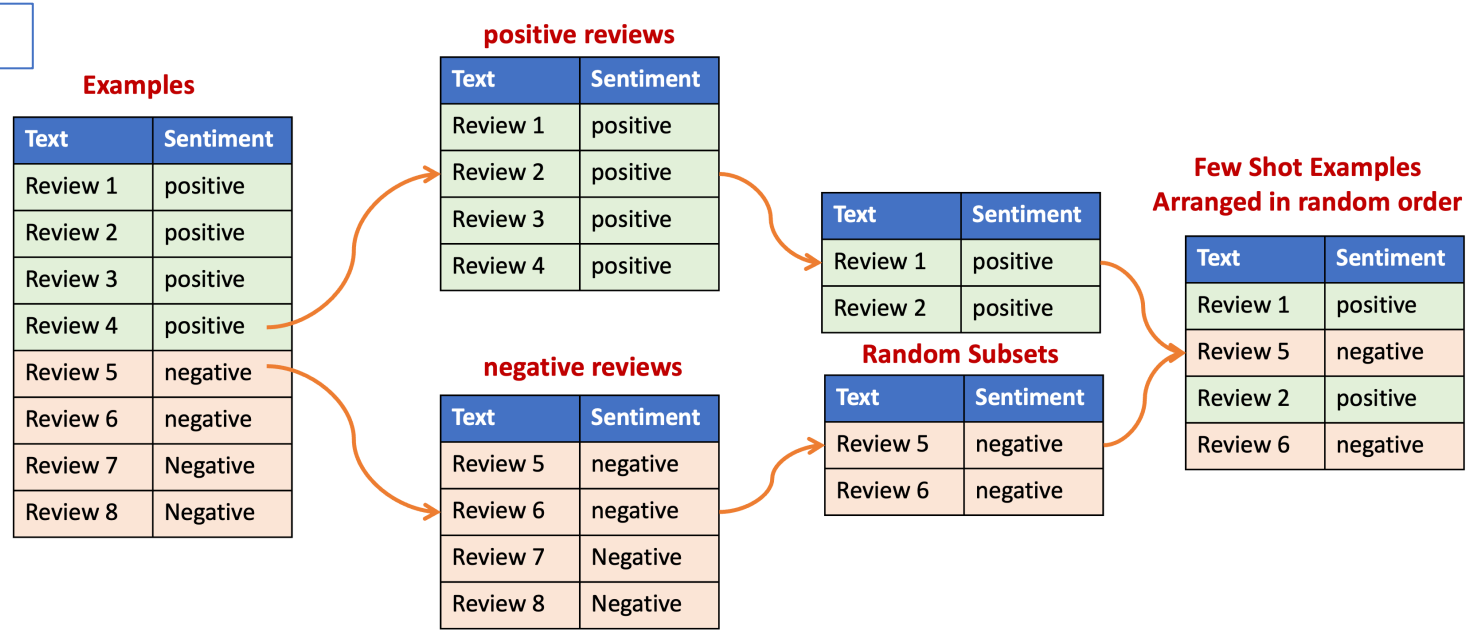
- Créons une fonction Python qui génère des exemples sans biais (notre fonction implémente le flux de travail présenté ci-dessous) :

```python
def create_examples(dataset, n=4):
    positive_reviews = (dataset.sentiment == 'positive')
    negative_reviews = (dataset.sentiment == 'negative')
    columns_to_select = ['text', 'sentiment']
    positive_examples = dataset.loc[positive_reviews, columns_to_select].sample(n)
    negative_examples = dataset.loc[negative_reviews, columns_to_select].sample(n)
    examples = pd.concat([positive_examples, negative_examples])
    # sampling without replacement is equivalent to random shuffling
    randomized_examples = examples.sample(2*n, replace=False)
    return randomized_examples.to_json(orient='records')
```

```python
examples = create_examples(examples_df, 2)
```

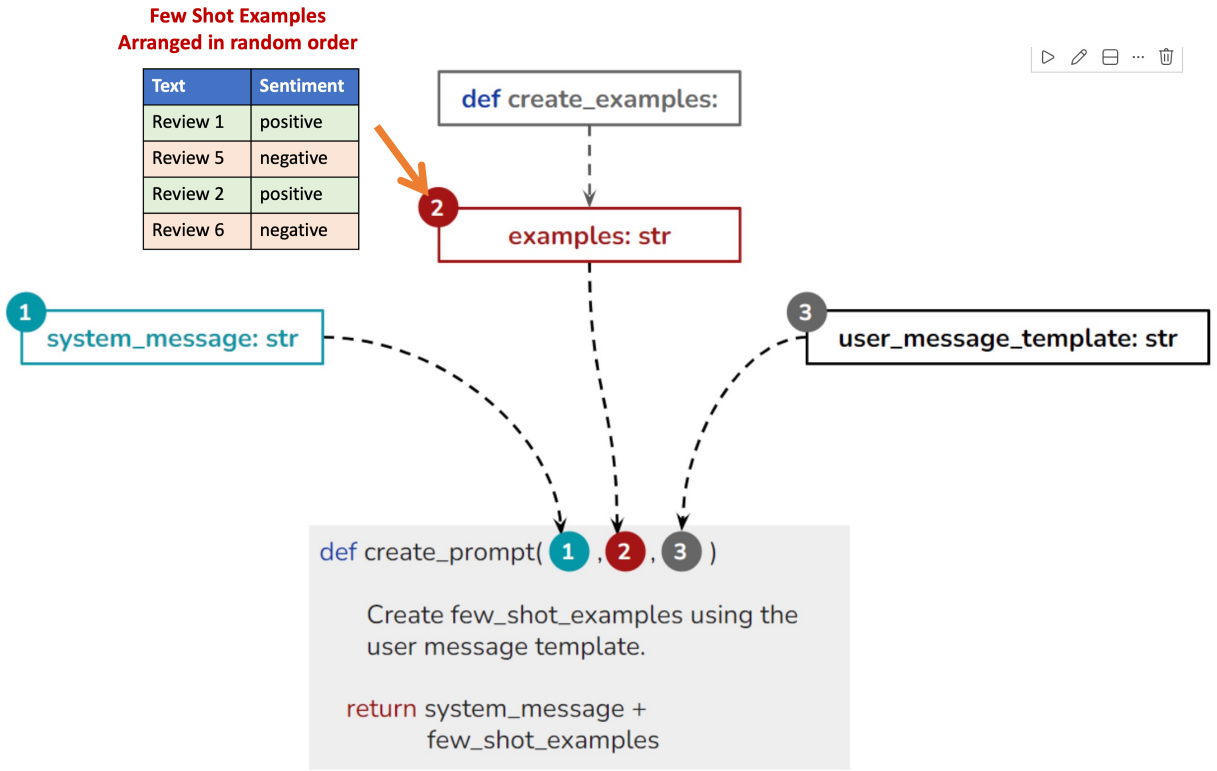
```python
json.loads(examples)
```

```
[{'text': 'Otto Preminger, completing a noir cycle at Tw
  'sentiment': 'positive'},
 {'text': 'This is one of the most irritating, nonsensic
  'sentiment': 'negative'},
 {'text': "<br /><br />One of the best films I've ever s
  'sentiment': 'positive'},
 {'text': 'First, IFC runs Town and Country, and now thi
  'sentiment': 'negative'}]
```



Étape 3 : Élaborer le prompt

- Une fois les exemples en place, nous pouvons désormais assembler un prompt **few-shot**.
- Comme nous utiliserons ce prompt **few-shot** à plusieurs reprises lors de l'évaluation, écrivons une fonction pour créer un prompt **few-shot** (la logique de cette fonction est illustrée ci-dessous).




```
def create_prompt(system_message, examples,
user_message_template):
    few_shot_prompt = [
              {'role':'system',
               'content': system_message}]
    for example in json.loads(examples):
        example_review = example['text']
        example_sentiment = example['sentiment']
        few_shot_prompt.append(
            {
                'role': 'user',
                'content': user_message_template.format(
                    movie_review=example_review
                )
            }
        )
        few_shot_prompt.append(
            {
                'role': 'assistant',
                'content': f"{example_sentiment}"}
        )
    return few_shot_prompt
```

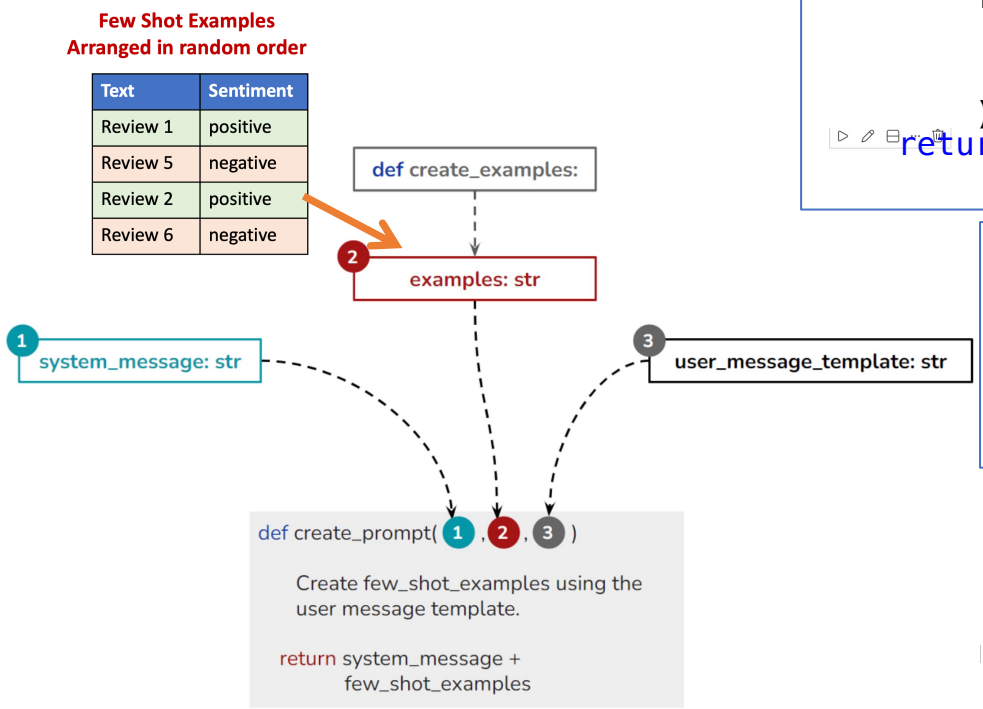
```
few_shot_pro
    few_shot_
    examples,
    user_messa
)
```

```
[{'role': 'system',
  'content': "\nClassify the sentiment of movie r
 {'role': 'user',
  'content': '```Otto Preminger, completing a noi
 {'role': 'assistant', 'content': 'positive'},
 {'role': 'user',
  'content': '```This is one of the most irritati
 {'role': 'assistant', 'content': 'negative'},
 {'role': 'user',
  'content': "```<br /><br />One of the best film
 {'role': 'assistant', 'content': 'positive'},
 {'role': 'user',
  'content': '```First, IFC runs Town and Country
 {'role': 'assistant', 'content': 'negative'}]
```

Étape 3 : Élaborer le prompt

- Une fois les exemples en place, nous pouvons désormais assembler un prompt **few-shot**.
- Comme nous utiliserons ce prompt **few-shot** à plusieurs reprises lors de l'évaluation, écrivons une fonction pour créer un prompt **few-shot** (la logique de cette fonction est illustrée ci-dessous).

```
def create_prompt(system_message, examples,
user_message_template):
    few_shot_prompt = [
                {'role':'system',
                 'content': system_message}]
    for example in json.loads(examples):
        example_review = example['text']
        example_sentiment = example['sentiment']
        few_shot_prompt.append(
            {
             'role': 'user',
             'content': user_message_template.format(
                    movie_review=example_review
                )
            }
        )
        few_shot_prompt.append(
            {
                'role': 'assistant',
                'content': f"{example_sentiment}"}
        )
    return few_shot_prompt
```




```
few_shot_prompt = create_prompt(
    few_shot_system_message,
    examples,
    user_message_template
)
```

User Input

```
{
    "role": "user",
    "content": "```User Review```"
}
```

Few Shot Prompt

System Message

```
{
 "role": "system",
 "content": "SYSTEM MESSAGE"
},
```

Few Shot Examples

```
{
 "role": "user",
 "content": "```Review 1```"
},
{
 "role": "assistant",
 "content": "positive"
},
{
 "role": "user",
 "content": " ```Review 2``` "
},
{
 "role": "assistant",
 "content": "negative"
},
```

```
]
```

Étape 3 : Élaborer le prompt

- **Prompt 3: Chain-of-Thought**
  - Pour le prompt **CoT** (**Chain-of-Thought**), nous ajoutons des instructions détaillées étape par étape au message système **few-shot**, en demandant au modèle de réfléchir soigneusement avant d'attribuer un sentiment.
  - Mis à part cet ajout, il n'y a pas d'autres changements par rapport au prompt **few-shot**.
  - Notez que les exemples restent les mêmes tandis que le message système change.

```
cot_system_message = """
Classify the sentiment of movie reviews presented in the input
as 'positive' or 'negative'
Movie reviews will be delimited by triple backticks ``` in the
input.
Answer only 'positive' or 'negative'
Do not explain your answer.

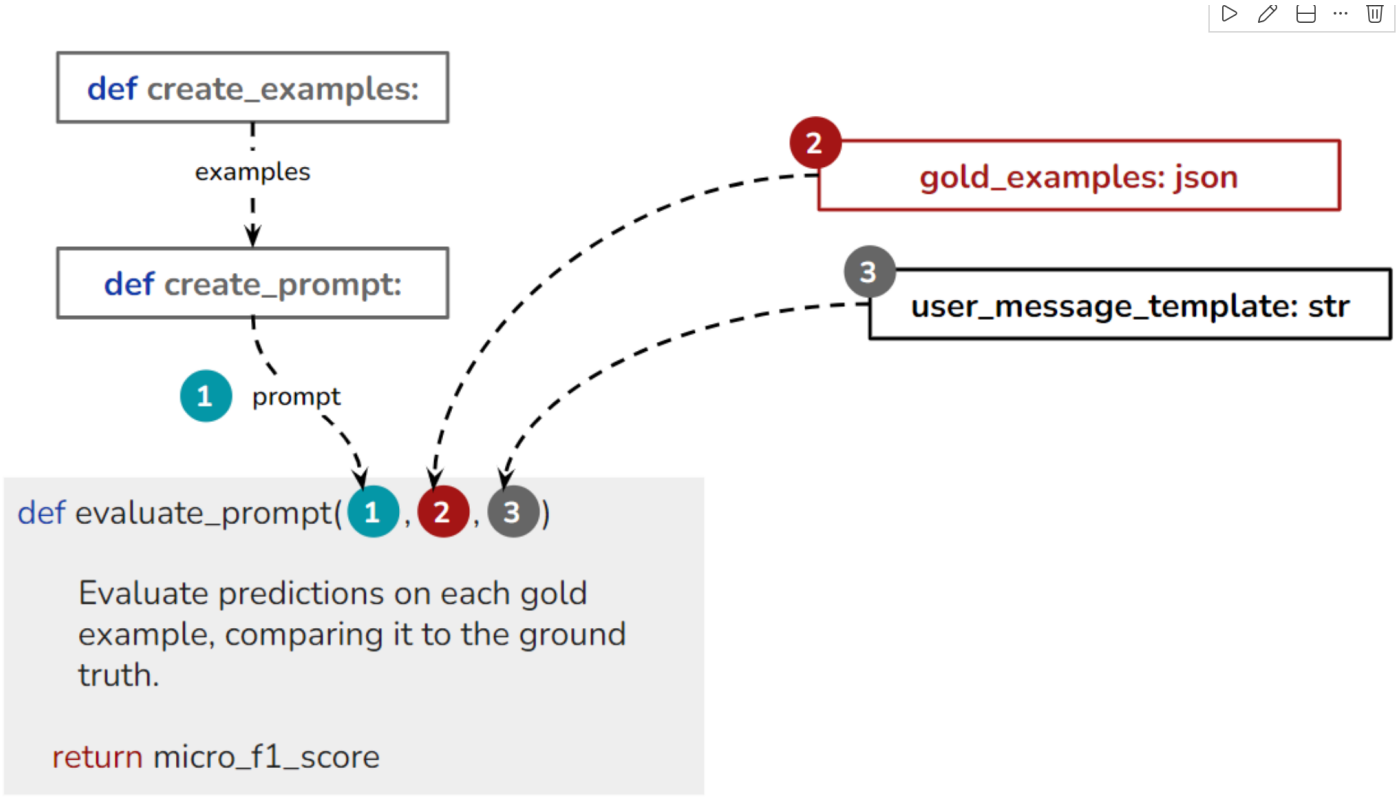
Instructions:
1. Carefully read the text of the review and think through the
options for sentiment provided
2. Consider the overall sentiment of the review and estimate
the probability of the review being positive

To reiterate, your answer should strictly only contain the
label: positive or negative
"""
```

```
cot_few_shot_prompt = create_prompt(
    cot_system_message,
    examples,
    user_message_template
)
```

Step 4 : Evaluate Prompts

- Nous avons maintenant trois ensembles de prompts à évaluer à l'aide des étiquettes de référence (**gold labels**).
- Étant donné que le prompt **few-shot** et le prompt **CoT** dépendent de l'échantillon d'exemples tiré pour constituer le prompt, nous nous attendons à une certaine variabilité dans l'évaluation.
- Nous évaluons donc chaque prompt plusieurs fois afin d'avoir une idée de la moyenne et de la variation autour de cette moyenne.
- Pour le redire, le choix d'un prompt doit tenir compte de la variabilité due au tirage aléatoire de l'échantillon.
- Pour faciliter les évaluations répétées, nous mettons en place une fonction d'évaluation (la logique globale est illustrée dans la figure ci-dessous).



Step 4 : Evaluate Prompts

- Utilisons maintenant cette fonction pour effectuer une évaluation des trois prompts assemblés jusqu'ici, en calculant à chaque fois le score **micro-F1**.

```
gpt5 = ChatOpenAI(
 model="gpt-5.2", temperature=0
)
```

```
evaluate_prompt(
 zero_shot_prompt,gold_examples,
 user_message_template, gpt5)
```

1.0

```
evaluate_prompt(
 few_shot_prompt,gold_examples,
 user_message_template, gpt5)
```

1.0

```
evaluate_prompt(
 cot_few_shot_prompt,gold_examples,
 user_message_template, gpt5)
```

1.0

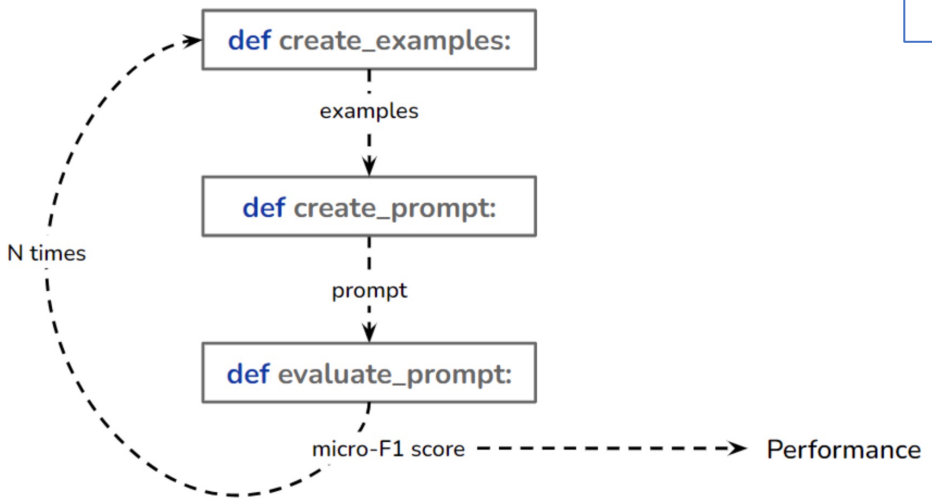
```
def evaluate_prompt(prompt, gold_examples, user_message_template, model):
    model_predictions, ground_truths = [], []

    for example in json.loads(gold_examples):
        gold_input = example['text']
        user_input = [
            {
                'role':'user',
                'content': user_message_template.format(movie_review=gold_input)
            }
        ]
        try:
            response = model.invoke(
               prompt+user_input
            )
            if hasattr (response, 'content'):
                response_content = response.content
            else:
                response_content = response
            if 'negative' in response_content.strip().lower():
                prediction='negative'
            elif 'positive' in response_content.strip().lower():
                prediction='positive'
            else:
                prediction ='unknown'
            model_predictions.append(prediction)
            ground_truths.append(example['sentiment'])
            #print(prediction, example['sentiment'])

        except Exception as e:
            print(e)
            continue
    micro_f1_score = f1_score(ground_truths, model_predictions, average="micro")
    return micro_f1_score
```

Step 4 : Evaluate Prompts

- Cependant, il ne s'agit que **d'un seul** choix d'exemples.
- Nous devons exécuter ces évaluations avec plusieurs choix d'exemples afin de nous faire une idée de la variabilité du score F1 pour le prompt **few-shot** et le prompt **CoT**.
- À titre d'exemple, exécutons les évaluations pour le prompt **few-shot** et le prompt **CoT 10** fois.




```
num_eval_runs = 10
few_shot_performance, cot_few_shot_performance = [], []
for _ in tqdm(range(num_eval_runs)):
    # For each run create a new sample of examples
    examples = create_examples(examples_df)
    # Assemble the few shot prompt with these examples
    few_shot_prompt = create_prompt(
       few_shot_system_message,
       examples, user_message_template)
    # Evaluate prompt accuracy on gold examples
    few_shot_micro_f1 = evaluate_prompt(
       few_shot_prompt,
       gold_examples,
       user_message_template, gpt5)
    few_shot_performance.append(few_shot_micro_f1)
    cot_few_shot_performance.append(few_shot_micro_f1)
```

```
100%|██████████| 10/10 [02:32<00:00, 15.22s/it]
```

```
   mean_val =np.array(few_shot_performance).mean()
   std_val= np.array(few_shot_performance).std()
```

```
   print(f"f1_score Moyen= {mean_val}, Standard Deviation = {std_val} ")
```

```
f1_score Moyen= 1.0, Standard Deviation = 0.0
```